Data Visualization & Multimodal Data

Matplotlib, Seaborn, Charts & Graphical Interpretation

Python Workshop

IIST

October 7, 2026

Content

1. Why Data Visualization?
2. Matplotlib Fundamentals
3. Chart Types & When to Use Them
4. Seaborn for Statistical Visualization
5. Multimodal Data & Multiple Visualizations
6. Graphical Interpretation & Best Practices
7. Summary

The Power of Visualization

Data vs. Visualization

- 1000 data points: Hard to interpret
- Same data in a chart: Insights in seconds
- Brain processes images thousands of times faster than text

Key Benefits

- Identify patterns & trends
- Communicate findings clearly
- Spot outliers & anomalies
- Make data-driven decisions

Which Is Easier to Understand?

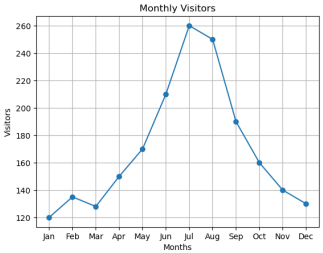
Monthly visitors to a park (hundreds)

A. A table

Jan	Feb	Mar	Apr	May	Jun
120	135	128	150	170	210
Jul	Aug	Sep	Oct	Nov	Dec
260	250	190	160	140	130

Which month was busiest?
Is there a seasonal pattern?

B. The same data as a chart



The numbers are the same. The chart shows the summer peak at a glance.

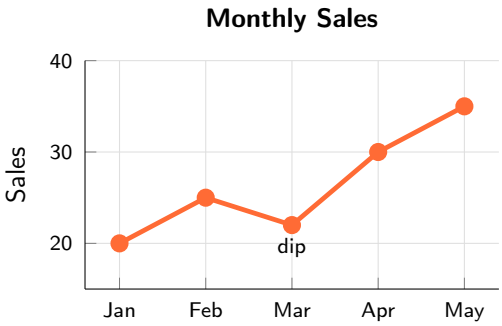
Why Do We Visualize Data?

Visualization helps us:

- See patterns
- Compare values
- Identify trends
- Detect unusual values
- Communicate clearly
- Support decisions

A table *stores* information.
A good chart helps people *understand* it.

From Raw Data to Insight



Raw data Jan–May: 20, 25, 22, 30, 35

↓

Visualization line chart (left)

↓

Pattern values mostly rise; March is lower

↓

Insight Sales show an overall increasing trend, with a dip in March.

Next question: What might explain the March dip?

The chart shows *what* happened, not *why*. Explaining it needs more information.

Applications in Real Life

- **Business:** Sales trends, customer behavior analysis
- **Healthcare:** Patient data, disease spread tracking
- **Security:** Threat patterns, operational metrics
- **Weather:** Meteorological data interpretation
- **Sports:** Performance analytics, game statistics

"A picture is worth a thousand words"

What Is Matplotlib?

Matplotlib is a Python library for creating charts and plots.

The basic workflow

1. Import the library
2. Prepare the data
3. Choose a plot
4. Add title and labels
5. Display the plot

Install (once):

```
pip install matplotlib
```

Import (every program):

```
import matplotlib.pyplot as plt
```

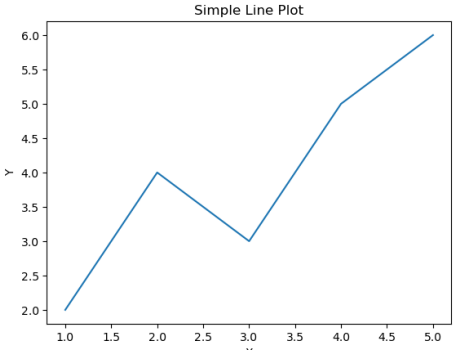
`plt` is just a short nickname.

Basic Plot Structure

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 3, 5, 6]

plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



Understanding the Code

Line	What it does
`import matplotlib.pyplot as plt`	Load the plotting tools
`x = [1, 2, 3, 4, 5]`	Positions along the x-axis
`y = [2, 4, 3, 5, 6]`	Values on the y-axis
`plt.plot(x, y)`	Draw a line through the points
`plt.title(...)`	Add a title
`plt.xlabel(...)`, `plt.ylabel(...)`	Label the axes
`plt.show()`	Display the finished chart

Customizing Plots

```
plt.plot(x, y,
color='red',              # Color
linestyle='--',           # Dashed line
linewidth=2,              # Line thickness
marker='o',               # Add markers -
    circle shape
markersize=8,             # Marker size
label='Data Series',      # Legend label
alpha=0.7)                # Transparency

plt.legend()                  # Show legend
plt.tight_layout()            # Auto-fit
    layout
```

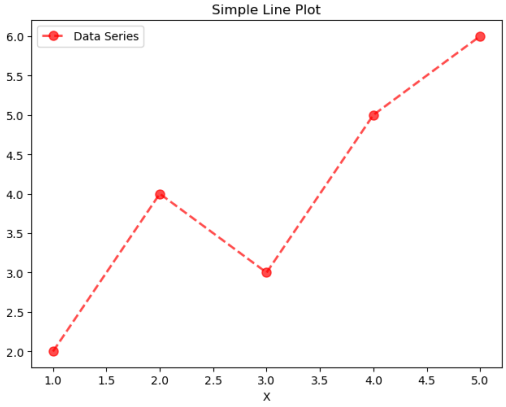


Figure vs Axes

Figure: The whole Canvas

- Container for all elements
- Size of the entire plot
- Can contain multiple subplots

Axes: The actual plot area

- Contains data, labels, ticks
- Can have multiple axes in one figure

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "
    May", "Jun"]
visitors = [120, 135, 128, 150, 170,
    210]

# Figure = complete canvas
# Axes   = actual plotting area
fig, ax = plt.subplots(figsize=(10, 6))

# Draw the graph inside the Axes
ax.plot(months, visitors, marker='o',
    linewidth=2)
```

```
# Customize the Axes
ax.set_title("Monthly Visitors",
    fontsize=16)
ax.set_xlabel("Months", fontsize=12)
ax.set_ylabel("Visitors", fontsize=12)

# Add grid
ax.grid(True, linestyle='--', alpha
    =0.5)

# Adjust spacing
fig.tight_layout()

# Display the Figure
plt.show()
```

Common Chart Types

1. **Line Plot**
   - Show trends over time
   - Multiple series comparison
2. **Bar Chart**
   - Compare categories
   - Discrete values
3. **Scatter Plot**
   - Relationship between variables
   - Identify clusters
4. **Histogram**
   - Distribution of data
   - Frequency analysis
5. **Heatmap**
   - 2D data visualization
   - Correlation patterns
6. **Box Plot**
   - Statistical summary
   - Outlier detection

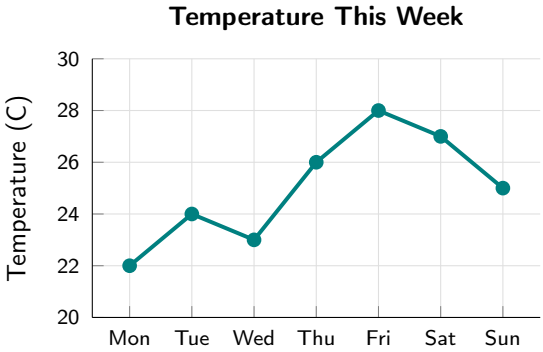
Which Chart Should I Use?

Chart	Use it to show...	Example
Line plot	change over time (a trend)	temperature each day
Bar chart	a comparison of categories	students per department
Scatter plot	a relationship between two numbers	hours studied vs score
Histogram	how values are spread out	exam marks of a class
Box plot	spread + unusual values, by group	marks of 3 classes
Heatmap	a table of values as colours	temperature by city & month

First ask: **"What question am I trying to answer?"**
The question decides the chart.

Line Plot: Change Over Time

Question: How did the temperature change during the week?

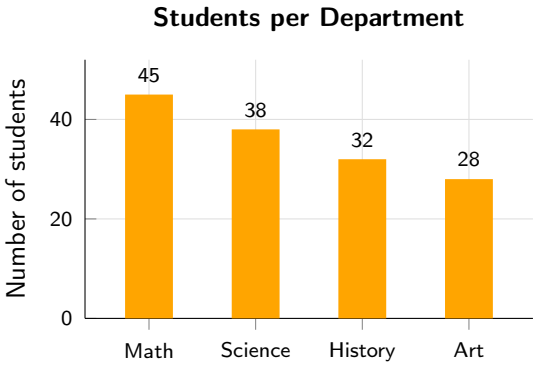


Key idea: use a line when the *order* of the x-values matters, e.g. time.

```
plt.plot(days, temp, marker="o",
         color="teal")
```

Bar Chart: Comparing Categories

Question: Which department has the most students?

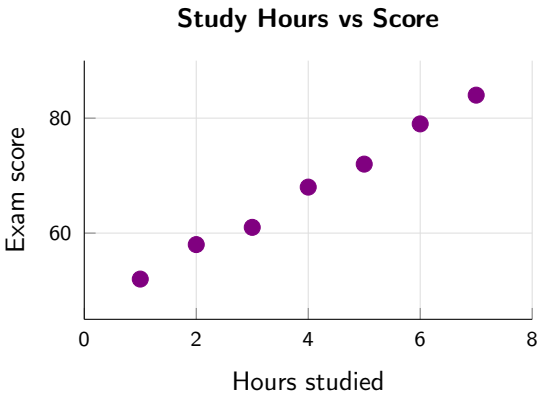


Key idea: the *height* of each bar is its value. Easy to see biggest and smallest.

```
plt.bar(depts, students,
        color="orange")
```

Scatter Plot: Is There a Relationship?

Question: Do students who study longer score higher?

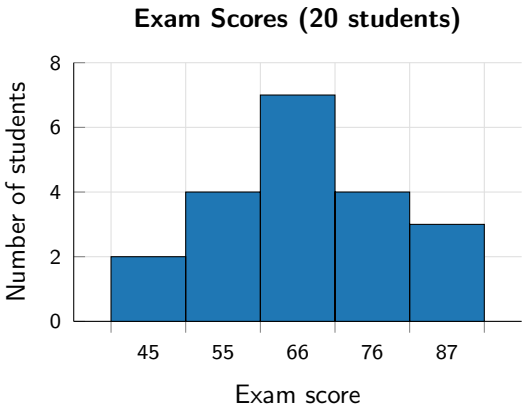


Key idea: one dot = one student. Dots rising to the right = the two values go up together.

```
plt.scatter(hours, score,
            color="purple")
```

Histogram: How Are Values Spread Out?

Question: How did the class score on the exam?



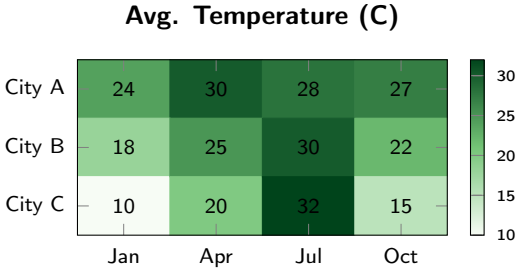
Key idea: values are grouped into ranges (**bins**); bar height = how many.

```
plt.hist(scores, bins=5,
         edgecolor="black")
```

Heatmap

Values as colours: good for spotting patterns in a matrix of values (e.g. a correlation matrix)

```
sns.heatmap(data, annot=True,
            cmap="Greens")
```

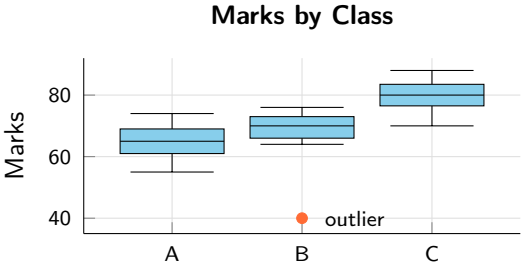


Box Plot

Box plot

Distribution & spread: median (line), quartiles (box), whiskers, possible outliers (dots)

```
sns.boxplot(x=groups, y=marks,
            color="skyblue")
```



Which Chart Would You Choose?

Think first, then answer:

1. Temperature over 7 days
2. Number of students in different departments
3. Height vs weight of students
4. Distribution of exam marks
5. Compare mark distributions across several classes

Which Chart Would You Choose?

Think first, then answer:

1. Temperature over 7 days → **Line plot**
2. Number of students in different departments → **Bar chart**
3. Height vs weight of students → **Scatter plot**
4. Distribution of exam marks → **Histogram**
5. Compare mark distributions across several classes → **Box plot**

Same rule every time: the **question** decides the chart.

Introduction to Seaborn

- Built on top of Matplotlib
- Focuses on statistical visualization
- Better default styles
- Works seamlessly with Pandas DataFrames

Installation:

```
pip install seaborn pandas
```

Basic Setup:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Seaborn vs Matplotlib

Matplotlib

- Flexible
- Detailed customization
- Steeper learning curve

Seaborn

- Statistical plots
- Better defaults
- Works with DataFrames
- Easier for beginners

Both are useful. Seaborn makes many common statistical charts easier; you can still use `plt.title()` etc. on a Seaborn chart.

Functions to know

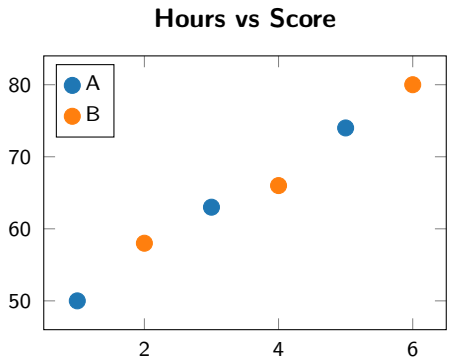
sns.lineplot()	line
sns.scatterplot()	scatter
sns.barplot()	bars (averages)
sns.histplot()	histogram
sns.boxplot()	box plot
sns.heatmap()	heatmap

Seaborn Example: Code

```
import seaborn as sns
import matplotlib.pyplot as plt

hours = [1, 2, 3, 4, 5, 6]
score = [50, 58, 63, 66, 74, 80]
group = ["A", "B", "A", "B", "A", "B"]

sns.scatterplot(x=hours, y=score,
                hue=group)
plt.title("Hours vs Score")
plt.show()
```



`hue=group`: colour each point by its group. Seaborn adds the colours *and* the legend for you.

Seaborn Plot Types

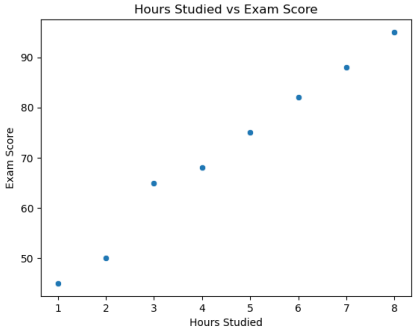
Line plot

```
sns.lineplot(x=months, y=visitors,
    marker="o")
```



Scatter Plot

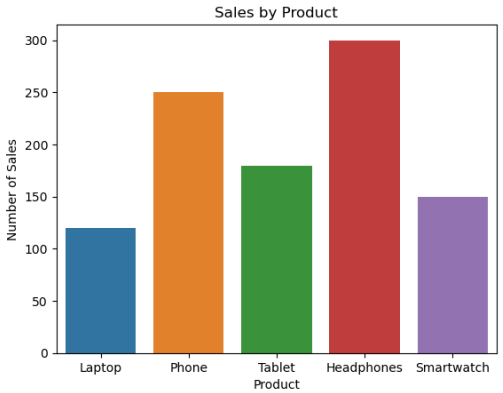
```
sns.scatterplot(x=hours, y=scores)
```



Seaborn Plot Types

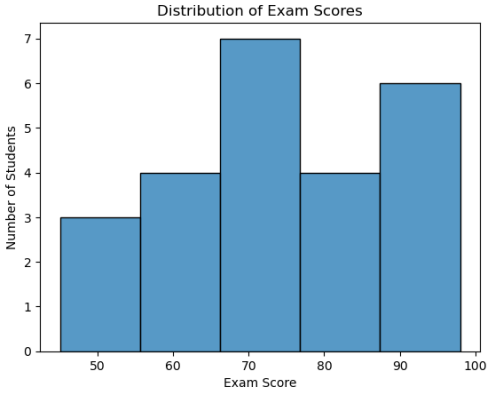
Bar Graph

```
sns.barplot(x=products, y=sales)
```



Histogram

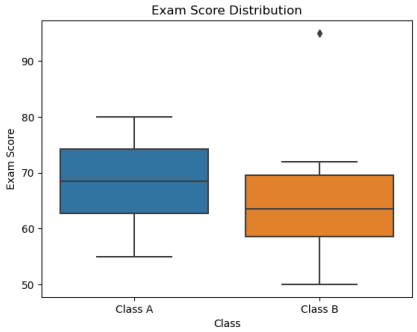
```
sns.histplot(scores, bins=5)
```



Seaborn Plot Types

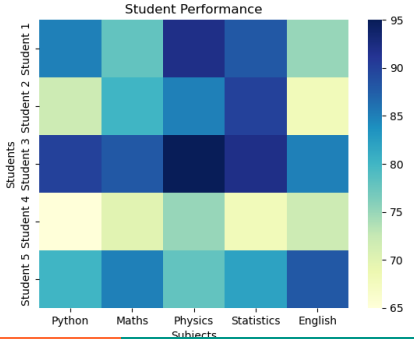
Box Plot

```
sns.boxplot(x=classes, y=scores)
```



Heatmap

```
sns.heatmap(scores, xticklabels=
    subjects, yticklabels=students,
    cmap="YlGnBu" )
```



What Is Multimodal Data?

Multimodal data = information that comes in multiple *forms/ modes/ types/ format*.

- Text + images
- Numbers + images
- Sensor readings + timestamps
- GPS location + measurements
- Audio + text
- Image + metadata

Requires multiple visualization approaches

What is Multimodal Data?

It means *different kinds of data*, and each kind may need a different way to show it.

Visualization Strategies:

- Subplots (multiple plots together)
- Combination plots (mixing plot types)
- Dual-axis plots (different scales)
- Dashboard-style layouts

Subplots Layout

```
fig, axes = plt.subplots(2, 2)

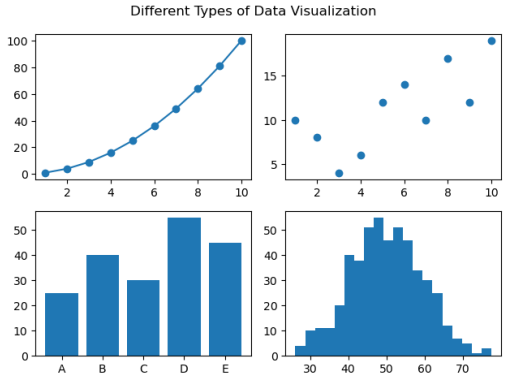
# Plot 1: Line Plot
axes[0, 0].plot(x, y1, marker="o")

# Plot 2: Scatter Plot
axes[0, 1].scatter(x, y2)

# Plot 3: Bar Plot
axes[1, 0].bar(categories, values)

# Plot 4: Histogram
axes[1, 1].hist(data, bins=20)

fig.suptitle("Different Types of Data
    Visualization")
```



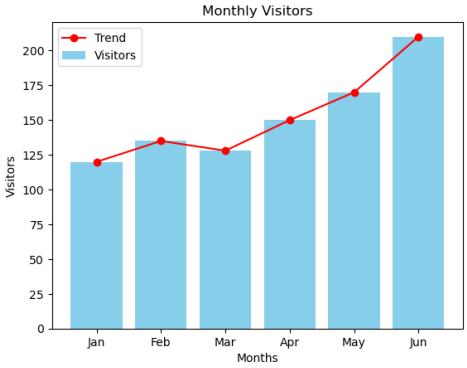
Combination plots

```
fig, ax = plt.subplots(figsize=(10, 6))

# Bar chart
ax.bar( months, visitors, color="skyblue
    ", label="Visitors" )

# Line plot
ax.plot( months, visitors, color="red",
    marker="o", linewidth=2, label="
    Trend" )

ax.set_title("Monthly Visitors")
ax.set_xlabel("Months")
ax.set_ylabel("Visitors")
ax.legend()
plt.show()
```

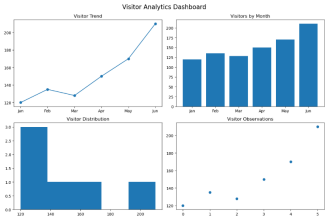


Dashboard-style Layout

```
fig, ax = plt.subplots(2, 2, figsize=(12, 8))

# 1. Line plot
ax[0, 0].plot(months, visitors, marker="o")
ax[0, 0].set_title("Visitor Trend")
# 2. Bar plot
ax[0, 1].bar(months, visitors)
ax[0, 1].set_title("Visitors by Month")
# 3. Histogram
ax[1, 0].hist(visitors, bins=5)
ax[1, 0].set_title("Visitor Distribution")
# 4. Scatter plot
ax[1, 1].scatter(range(len(months)), visitors)
ax[1, 1].set_title("Visitor Observations")

fig.suptitle("Visitor Analytics Dashboard", fontsize=16)
plt.show()
```

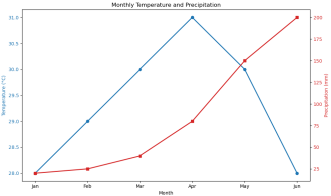


Dual-Axis Plot

```
# Two different scales
fig, ax1 = plt.subplots(figsize=(10, 6))

# First axis (left)
color = 'tab:blue'
ax1.set_xlabel('Month')
ax1.set_ylabel('Temperature (C)', color=color)
ax1.plot(months, temperature, color=color, marker='o')
ax1.tick_params(axis='y', labelcolor=color)

# Second axis (right)
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Precipitation (mm)', color=color)
ax2.plot(months, precipitation, color=color, marker='s')
ax2.tick_params(axis='y', labelcolor=color)
```



One Observation, Many Modes

One weather observation, stored as a Python dictionary:

```
observation = {
    "time": "09:00",
    "temperature": 27.5,
    "location": (10.0, 76.3),
    "sky": "light rain",
    "image": "satellite.png",
}
for key, value in observation.items():
    print(key, type(value).__name__)
```

Output

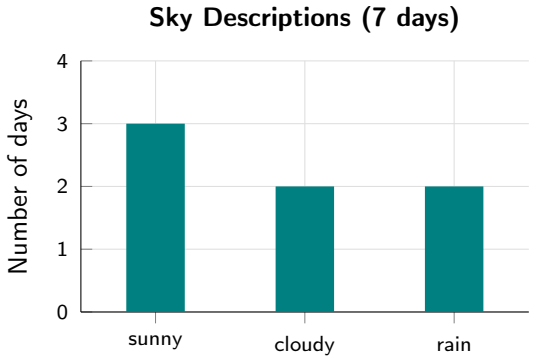
```
time str
temperature float
location tuple
sky str
image str
```

Five values, **five modes**: time, number, GPS position, text, image.

Each mode needs its own way to be shown.

Text Mode: Count the Words, Then Plot

Question: What was the sky like this week?

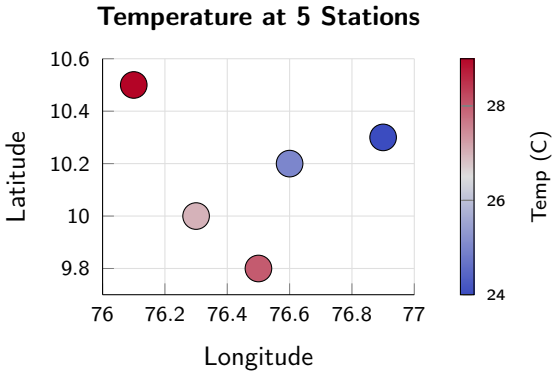


Key idea: we can't plot words directly, so we *count* them first.

```
from collections import Counter
sky = ["sunny", "cloudy", "rain",
       "sunny", "rain", "sunny",
       "cloudy"]
counts = Counter(sky)
plt.bar(counts.keys(), counts.values(),
        color="teal")
plt.title("Sky Descriptions (7 days)")
plt.ylabel("Number of days")
plt.show()
```

GPS + Measurement: A Simple Map

Question: Where was it warmest?

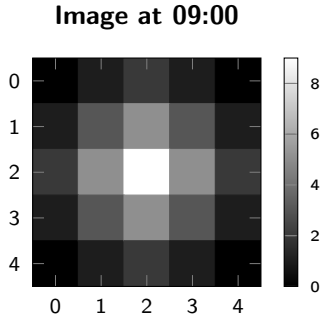


Key idea: GPS gives the *position*; the measurement gives the *colour*.

```
lat  = [10.0, 10.2, 10.5, 9.8, 10.3]
lon  = [76.3, 76.6, 76.1, 76.5, 76.9]
temp = [27, 25, 29, 28, 24]
plt.scatter(lon, lat, c=temp, s=150,
            cmap="coolwarm")
plt.colorbar(label="Temp (C)")
plt.title("Temperature at 5 Stations")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()
```

Image + Metadata

Question: How do we show an image together with its information?



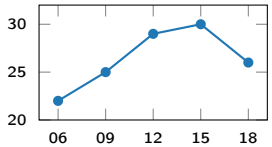
Real photo: `plt.imread("photo.png")`

Key idea: an image is a grid of numbers (brightness). The metadata becomes the title.

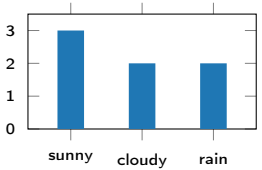
```
image = [[0, 1, 2, 1, 0],
         [1, 3, 5, 3, 1],
         [2, 5, 9, 5, 2],
         [1, 3, 5, 3, 1],
         [0, 1, 2, 1, 0]]
info = {"time": "09:00"}
plt.imshow(image, cmap="gray")
plt.title("Image at " + info["time"])
plt.colorbar()
plt.show()
```

Putting the Modes Together: Subplots

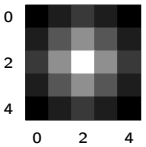
Temperature: number + time



Sky: text



Satellite: image




```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].plot(hours, temps, marker="o")    # number + time
axes[1].bar(counts.keys(), counts.values())  # text, counted
axes[2].imshow(image, cmap="gray")        # image
plt.tight_layout()
plt.show()
```

How to Read Charts

Line Plot:

- Direction = Trend (up/down)
- Slope = Rate of change
- Gaps = Missing data or anomalies

Bar Chart:

- Height = Magnitude
- Compare adjacent bars
- Look for patterns across groups

Scatter Plot:

- Clustering = Groups/patterns
- Spread = Correlation strength
- Outliers = Unusual points

Visualization Best Practices

1. **Choose the Right Chart:** Match chart type to your message
2. **Keep it Simple:** Avoid clutter and unnecessary elements
3. **Use Color Wisely:** Highlight important information
4. **Label Everything:** Clear titles, axis labels, legends
5. **Tell a Story:** Your plot should communicate a message
6. **Check for Accuracy:** Verify data and scaling

Common Mistakes to Avoid

- × Too many elements in one plot
- × Poor color choices (hard to distinguish)
- × Missing or unclear labels
- × Wrong chart type for your data
- × Misleading scales or truncation
- × 3D plots when 2D works better

Key Takeaways

- Visualization turns raw data into insights
- Matplotlib: Powerful, flexible, control-oriented
- Seaborn: Statistical, elegant, DataFrame-friendly
- Choose the right chart for your data
- Label everything and keep it simple